SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	**Assessment:**	Constraint Based Problem Solving
Weightage:	40%	**Total Marks:**	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	LAKSHMI DEVI .J	**Reg. No.:**	192524300
Section / Batch:	2025	**Date:**	25.08.26

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

    - CPU utilization must be below 15%
    - Multiple sensors generate interrupts every 1 ms
    - Use vectored interrupt handling
    - Select between memory-mapped and I/O-mapped I/O for optimal latency

I/O Communication Mechanism for a Real-Time Embedded System

Given Constraints

- CPU utilization < 15%
- Multiple sensors generate interrupts every 1 ms
- Vectored interrupt handling required
- Select memory-mapped or I/O-mapped I/O for low latency

Proposed Architecture

Design

Use **memory-mapped I/O (MMIO)** because sensor registers can be accessed using normal load/store instructions. This generally provides simpler and faster register access than a separate I/O instruction space on architectures that support both.

A **vectored interrupt controller** assigns a unique interrupt vector to each sensor. When a sensor generates an interrupt, the CPU directly jumps to the corresponding Interrupt Service Routine (ISR).

Interrupt Handling

Sensor generates interrupt

↓

Interrupt Controller

↓

Identify interrupt vector

↓

Jump directly to ISR

↓

Read sensor register

↓

Store data in buffer

↓

Clear interrupt

↓

Return to main program

CPU Utilization

If an ISR takes approximately **10 µs** and a sensor generates one interrupt every **1 ms**:

$$CPU\ utilization = \frac{10\mu s}{1000\mu s} \times 100$$
$$= 1\%$$

Even with several sensors, the ISR should be kept extremely short. For example, with 10 equivalent interrupt sources:

$$10 \times 1\% = 10\%$$

which remains below the required **15%**.

Conclusion

Memory-mapped I/O + vectored interrupts + short ISRs provides low latency while keeping CPU utilization below 15%. Sensor processing that is not time-critical should be deferred to background tasks rather than performed inside the ISR.

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

High-Speed I/O Subsystem Using NVMe and PCIe

Given Constraints

- Storage throughput > 3 GB/s
- NVMe over PCIe
- CPU should not perform bulk data transfer
- DMA controller with interrupt notification

Working

1. CPU initializes the NVMe controller.
2. CPU creates NVMe submission queues.
3. The application requests a read/write operation.
4. NVMe controller performs the actual data transfer using **DMA**.
5. Data moves directly between NVMe storage and system memory.
6. CPU does not copy every block of data.
7. After completing the operation, the controller generates an interrupt.
8. CPU processes the completion notification.

DMA Operation

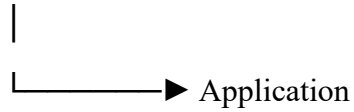
CPU

Issue command

NVMe Controller

DMA transfer

Main Memory



Throughput

PCIe bandwidth must be selected so that the usable bandwidth exceeds 3 GB/s.

For example, a PCIe 4.0 x4 NVMe interface provides substantially more than 3 GB/s of theoretical link bandwidth, leaving sufficient bandwidth for a storage device capable of >3 GB/s.

Advantages

- High storage throughput
- Very low CPU overhead
- Direct memory transfer
- Queue-based NVMe architecture
- Interrupt only after completion
- Suitable for high-performance systems

Conclusion

The optimal design is **NVMe over PCIe with DMA and interrupt-based completion notification**. DMA handles bulk data movement while the CPU handles only commands, queue management, and completion processing.

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

Multi-Device Communication Architecture

Given Constraints

- 6 USB peripherals
- 2 high-speed Thunderbolt devices
- Avoid interrupt starvation
- Hardware and software interrupts
- Fair bus arbitration

Interrupt Strategy

Use different priorities:

Device	Interrupt Priority
Thunderbolt 1	Very High
Thunderbolt 2	Very High
USB 1–6	Medium
Background software tasks	Low

High-speed Thunderbolt devices receive hardware interrupt priority because they can generate high-rate events.

USB events are handled through the USB controller rather than assigning excessive CPU attention to every low-level device event.

Avoiding Interrupt Starvation

Use:

- Priority-based interrupt handling
- Interrupt masking during critical sections
- Interrupt coalescing
- Round-robin scheduling for devices with equal priority
- Bounded ISR execution time

Fair Bus Arbitration

A **round-robin arbitration mechanism** can be used for devices of equal priority:

USB1 → USB2 → USB3 → USB4 → USB5 → USB6

The arbiter gives each requesting device a limited bus time.

Hardware and Software Interrupts

Hardware interrupts:

- USB controller events
- Thunderbolt events
- Device completion events

Software interrupts:

- Deferred processing
- Background driver tasks

- Operating-system service requests

Conclusion

The combination of **priority-based interrupts, interrupt coalescing, round-robin arbitration, hardware interrupts, and software interrupts** prevents starvation while providing high-speed communication for Thunderbolt devices.

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

Data Acquisition System

Given Constraints

- Continuous data stream = **500 MB/s**
- Minimal CPU intervention
- Compare DMA and interrupt-driven I/O
- Memory-mapped device registers

DMA vs Interrupt-Driven I/O

Feature	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Large data transfer	Excellent	Poor
500 MB/s stream	Suitable	Not suitable
Transfer overhead	Low	High
CPU utilization	Low	High
Continuous streaming	Excellent	Difficult
Recommended	**Yes**	No

DMA Operation

The CPU initially configures:

- Source address
- Destination address
- Transfer size

- Buffer descriptors
- DMA control registers

After configuration:

Sensor

↓

DAQ Device

↓

DMA Controller

↓

Memory Buffer

↓

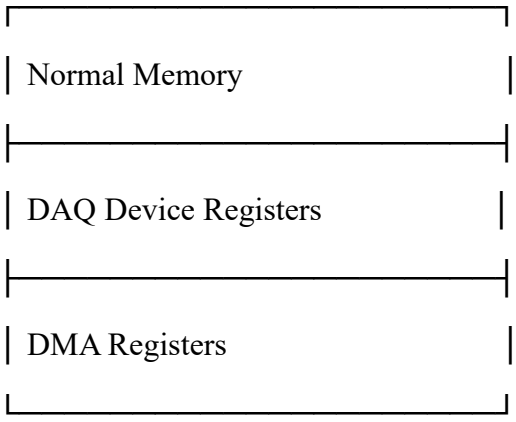
CPU notified by interrupt

The CPU receives an interrupt only when a buffer is filled or a DMA transfer reaches a defined completion point.

Why Memory-Mapped I/O?

Device registers are mapped into the processor's address space:

CPU Address Space



The CPU can configure the device using normal load/store operations.

Conclusion

For a continuous **500 MB/s** stream, **DMA is clearly preferable** to interrupt-driven I/O. Memory-mapped registers allow efficient device configuration, while DMA performs bulk transfers without continuous CPU intervention.

5.Construct an interrupt handling mechanism with constraints:

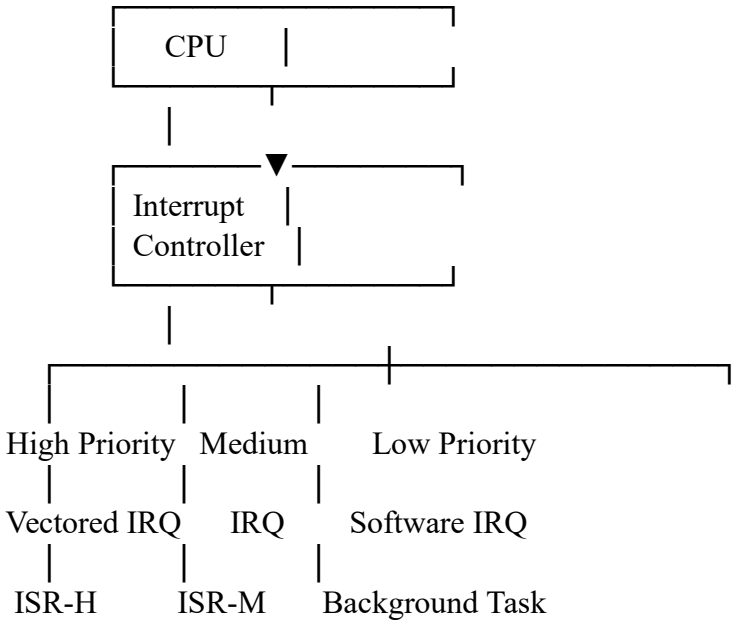
- Support nested interrupts
- Maximum interrupt latency < 10 μs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Interrupt Handling Mechanism

Given Constraints

- Nested interrupts
- Maximum interrupt latency < 10 μs
- Vectored interrupts for high-priority devices
- Software interrupts for background tasks

Architecture



Priority Levels

Priority	Type	Example
0	Highest	Emergency/high-speed device
1	High	Network/data acquisition
2	Medium	Normal peripherals
3	Low	Background tasks

Nested Interrupt Operation

Suppose the CPU is executing a medium-priority ISR:

Main Program

↓

Medium Interrupt

↓

Medium ISR

↓

High-Priority Interrupt

↓

Save current context

↓

High-Priority ISR

↓

Restore medium ISR

↓

Continue Medium ISR

↓

Return to Main Program

The high-priority interrupt is allowed to preempt the lower-priority ISR.

Vectored Interrupts

Each high-priority device has a unique interrupt vector:

Vector Table

Vector 0 → High-speed Device ISR
Vector 1 → Network ISR
Vector 2 → Timer ISR
Vector 3 → Peripheral ISR

Therefore, the processor can immediately branch to the appropriate handler rather than searching for the source.

Maintaining <10 µs Latency

To achieve the required maximum latency:

1. Keep ISRs very short.
2. Use vectored interrupt hardware.
3. Assign high priority to real-time devices.
4. Avoid long critical sections.
5. Use interrupt nesting.
6. Defer non-critical processing to background tasks.
7. Use interrupt masking only for very short critical sections.

Software Interrupts

Software interrupts are used for tasks that do not require immediate hardware response:

Hardware ISR

↓

Store event/data

↓

Schedule software interrupt

↓

Background processing

This prevents lengthy processing from increasing hardware interrupt latency.

Conclusion

A **priority-based vectored interrupt controller with nested interrupts** provides fast response for critical devices. High-priority hardware interrupts are serviced immediately, while software interrupts handle background processing, helping maintain the **<10 µs interrupt-latency requirement**.

Code of Conduct

I J. LAKSHMI DEVI (192524300) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator **Faculty Incharge**